

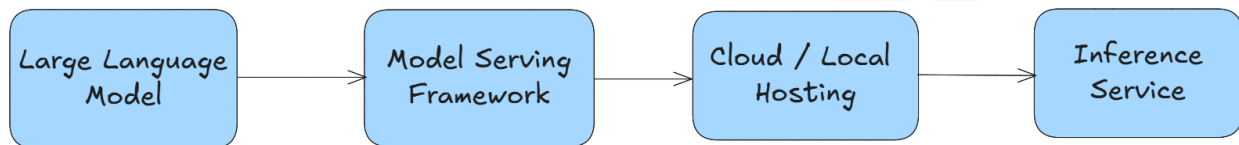


TMLC
ACADEMY

LLM Model Serving



Large Language Models (LLMs) have revolutionized industries, powering applications from chatbots to personalized recommendations. But, deploying these powerful models to deliver real-world impact requires an essential step: **model serving**. In this blog, we'll explore the nuances of model serving, diving into batch vs. online serving, server vs. serverless architectures, and the tools enabling LLM deployments.



What is Model Serving?

Model serving refers to the process of hosting and managing models so they can respond to requests or process data. It's akin to putting the model into production, enabling it to generate predictions or insights for end-users. Effective model serving ensures reliability, scalability, and efficiency while meeting latency and throughput requirements.

Batch vs. Online Model Serving

Model serving can broadly be categorized into two paradigms:

Batch Serving

In batch serving, predictions are generated for large datasets all at once. This approach is well-suited for scenarios like:

- Generating personalized recommendations for millions of users overnight.
- Processing data-intensive workflows like fraud detection on transaction logs.

Advantages:

- Cost-effective since computations can be scheduled during off-peak hours.
- Ideal for non-time-sensitive tasks.

Challenges:

- Not suitable for real-time applications.

Online Serving

Online (or real-time) serving processes requests as they come in, responding within milliseconds to seconds. It's commonly used in:

- Chatbots, where users expect instantaneous replies.
- Recommendation systems for e-commerce, adapting to user behaviour in real-time.

Advantages:

- Delivers low-latency responses for dynamic applications.
- Supports adaptive, interactive experiences.

Challenges:

- Higher infrastructure costs due to the need for 24/7 availability.
- Requires careful optimization to meet latency targets.

Server vs. Serverless Architectures

Another critical decision is the choice between server-based and serverless model serving. Let's examine their trade-offs.

Server-Based Model Serving

This approach involves hosting models on dedicated servers or virtual machines. AWS EC2 is one example of this.

Pros:

- Greater control over hardware and infrastructure.
- Suitable for high-throughput, consistent workloads.

Cons:

- Requires significant DevOps expertise.
- Potentially underutilized resources during low-traffic periods.

Serverless Model Serving

Serverless options abstract away the underlying infrastructure, scaling automatically based on demand. Services like AWS Lambda, Google Cloud Run, and Azure Functions fall into this category.

Pros:

- Pay-per-use pricing minimizes costs.
- Scales elastically with traffic.

Cons:

- Latency overhead from cold starts.
- Limited support for long-running processes or large model sizes.

Tools for LLM Model Serving

Server-Based Tools and Frameworks

1. MLflow

- Enables model versioning and deployment workflows.
- Supports integration with popular cloud platforms.

2. AWS SageMaker

- A comprehensive solution for training and deploying models at scale.
- Offers built-in monitoring and versioning.

3. Ollama

- Tailored for running and serving LLMs locally or on private servers.

4. vLLM

- Optimized for high-throughput and low-latency LLM inference.

5. Hugging Face TGI (Text Generation Inference)

- A robust tool designed for deploying Hugging Face models efficiently.

6. NVIDIA Triton Inference Server

- Optimized for high-performance inference on GPUs.
- Supports a variety of ML frameworks like TensorFlow, PyTorch, and ONNX.

7. Ray Serve

- A flexible, scalable solution designed for distributed serving.
- Ideal for complex workflows requiring coordination between multiple models.

8. BentoML

- Simplifies the packaging and deployment of models into production.

Serverless Tools

1. AWS / Azure / GCP Serverless Services

2. HuggingFace Inference Endpoint

- HuggingFace managed model serving

3. Cerebrum / Modal / Runpod:

- Streamlined serverless platforms for deploying and managing models with ease.
- Flexible serverless solutions tailored for ML/AI workloads.
- All these services provide the GPUs necessary for your workload without the need for you to manage them.

Key Considerations for LLM Model Serving

1. Scalability

- a. Can your infrastructure handle sudden spikes in traffic? Serverless architectures often shine here, but server-based options offer predictable performance.

2. Latency

- a. Real-time applications demand response times in milliseconds. Optimization can be possible by leveraging GPU acceleration, caching, or model quantization.

3. Cost Efficiency

- a. Batch processing can significantly reduce costs for non-critical workloads.
- b. Evaluate cloud pricing models to strike the right balance.

4. Model Updates

- a. How easily can you roll out updates or manage multiple versions?
- b. Tools like MLflow streamline version control and rollback.

Conclusion

LLM model serving is the linchpin for transforming cutting-edge research into impactful applications. By understanding batch vs. online serving, server-based vs. serverless architectures, and leveraging the right tools, businesses can unlock the full potential of LLMs. Whether you're building an AI-driven chatbot or a dynamic recommendation engine, thoughtful model serving is key to success.